Universidad Centroamericana José Simeón Cañas
Facultad de Ingeniería y Arquitectura



Informe proyecto

Materia:
Teoría de Lenguajes de Programación

Catedrático:
Jaime Roberto Clímaco Navarrete

Alumnos:
Marco Gabriel Arana Renderos 00037822
Natalia Melissa Avelar Menjívar 00135922
Cristofer Ricardo Díaz Alfaro 00071222
Ricardo Andrés García Barahona 00148922
Josué Fernando Gómez Guardado 00103722

Fecha de entrega:
Sábado 7 de noviembre de 2026

Marco teórico sobre jerarquía de Chomsky y limitaciones de las CFG

La jerarquía de Chomsky (Chomsky, 1956; Alfonseca et al., 2007) clasifica las gramáticas en cuatro tipos según la forma de sus reglas. Las gramáticas libres de contexto (CFG, Tipo 2) alcanzan para describir la sintaxis de un lenguaje de programación. Frente a ese lenguaje formal, el español tiene semejanzas que explican por qué vale la pena intentar una CFG, y diferencias que explican por qué se queda corta.

Tipo	Clase	Forma de las reglas	Autómata	Ejemplo en este proyecto
3	Regular	A -> aB, A -> a	Autómata finito	Identificadores y números del lexer
2	Libre de contexto (CFG)	A -> α, con un solo no terminal a la izquierda	Autómata de pila	Sintaxis de MiniC, $a^n b^n$, subordinadas en medio de la oración (español)
1	Sensible al contexto	αAβ -> αγβ, con γ ≠ ε	Autómata linealmente acotado	$a^n b^n c^n$, ww
0	Recursivamente enumerable	Sin restricciones	Máquina de Turing	Cualquier lenguaje semidecidible

Semejanzas

- **Estructura en árbol.** En ambos, las oraciones se agrupan en partes que contienen otras partes (una expresión dentro de una condición, un sintagma dentro de una oración). Por eso las dos se pueden representar con árboles de análisis.
- **Recursividad.** Ambos repiten una estructura dentro de sí misma: un bloque dentro de otro en MiniC, una subordinada dentro de otra en español (El profesor sabe que el niño dice que el perro corre).
- **Concordancia local.** La concordancia entre artículo y sustantivo, o entre sujeto y verbo, se puede escribir en una CFG, porque género y número tienen pocos valores: basta repetir las reglas por cada combinación (SN_MS, SN_FS...), que es lo que hicimos.

Diferencias

- **Ambigüedad.** En un lenguaje de programación la gramática se diseña para que cada programa tenga un solo árbol; por eso MiniC exige llaves en el if, para evitar el dangling else (Louden, 2004). En español una oración puede tener varios árboles: en La mujer vive en la casa de la ciudad, la frase de la ciudad puede describir a la casa o al verbo.
- **Dependencias de contexto.** En un lenguaje de programación, exigir que una variable esté declarada se deja al análisis semántico, que queda fuera de este trabajo. En el español, algunas dependencias, como emparejar dos listas de largo no acotado en el mismo orden (dependencias cruzadas), exceden a una CFG. El caso clásico es el alemán suizo (Shieber, 1985); para el español es discutido.

- **Vocabulario.** En MiniC el lexer reconoce cualquier identificador con una regla regular (Tipo 3). El español necesita un diccionario, y una palabra que no está en él (como astronauta) detiene al parser.

Descripción de la gramática del parser formal

Gramática del parser formal (MiniC)

MiniC es una versión reducida de C: declaraciones int, asignaciones, aritmética, seis comparaciones, if / else if / else, while y for, con cuerpos entre llaves. Las reglas viven en gramatica_minic.txt, separado del motor: para cambiar de lenguaje se edita ese archivo.

Decl -> INT ID DeclTail SEMI
If -> IF LPAREN Cond RPAREN Block ElsePart
ElsePart -> ELSE ElseTail | eps
ElseTail -> Block | If
Cond -> Expr RelOp Expr
Expr -> Term ExprP
ExprP -> PLUS Term ExprP | MINUS Term ExprP | eps
Term -> Factor TermP
TermP -> TIMES Factor TermP | DIVIDE Factor TermP | eps
Factor -> LPAREN Expr RPAREN | ID | NUM

Decisiones de diseño:

- **Sin recursión por la izquierda:** una regla como E → E + T se expandiría sin fin, así que se reescribe como E → T E'.
- **Precedencia por niveles:** Expr, Term y Factor hacen que * y / se agrupen antes que + y - .
- **Asociatividad:** el árbol anida ExprP hacia la derecha, por lo que no refleja que 10 - 4 - 3 se agrupa a la izquierda. Un evaluador tendría que recorrerlo de izquierda a derecha.
- **Llaves obligatorias:** sin ellas, ELSE estaría en FIRST y en FOLLOW de ElsePart (el dangling else), un conflicto LL(1).
- **Condición con comparación**: while (x) se rechaza.

Cálculo: Grammar obtiene FIRST (con qué token puede empezar un símbolo) y FOLLOW (qué token puede venir después) por punto fijo, y con ellos arma la tabla M[A, t]. Si una celda recibe dos reglas, lanza GrammarError. La gramática completa tiene 53 no terminales, 102 producciones y 177 celdas sin conflictos (22, 44 y 82 en la parte de MiniC).

Léxico y errores: PLY lee el identificador completo y luego consulta las reservadas, así integer es un ID. Se rechazan las palabras clave de C como nombres y los números con cero inicial. Si falta un símbolo, se registra y se sigue. Si no hay regla en la tabla, se descartan tokens hasta uno de FOLLOW (modo pánico). Cada error guarda línea y columna, en el orden del fuente.

Evidencia. 20 casos de aceptación y rechazo, 11 pruebas unitarias. El análisis semántico queda fuera del alcance.

Intentos de extensión al español

Se amplió la misma gramática: Program -> StmtList | Oracion. Las dos ramas no chocan porque empiezan con tokens distintos (INT, ID... frente a DET_*). Con --es, el lexer usa un diccionario de 161 palabras en 19 categorías cuyo nombre incluye género y número (DET_FS, SUST_MP, VERB_3S). Las oraciones se analizan desde "Oracion", para que una declaración de MiniC o un texto vacío no pasen por oración.

Fenómeno	Cómo se resuelve
Artículo, sustantivo y adjetivo	`SN_MS → DET_MS SUST_MS AdjMS`, y las otras tres combinaciones
Sujeto y verbo	`SV_MS` solo admite verbos singulares; `SV_MP`, solo plurales
Subordinada	`Cpl_MS → QUE Oracion` (recursiva)
Modificadores	Adjetivos pospuestos y `ModsS` (adverbios y frases preposicionales)

Resultados: Los 19 casos del español pasan. Una subordinada de dos niveles produce tres nodos Oracion. En la demo de 13 oraciones, el parser acepta 3 y rechaza 10. Contra la gramaticalidad real del español, su precisión es del 100 % (no aceptó ninguna oración incorrecta) y su cobertura del 37,5 % (3 de 8 correctas).

Puntos de falla

Oración	Regla que falla	Fenómeno	Nivel
La niño corre	`SN_FS` espera un sustantivo femenino	Concordancia artículo-sustantivo	Tipo 2 (rasgos finitos)
Los niños corre	Tras sujeto plural no cabe verbo singular	Concordancia sujeto-verbo	Tipo 2 (rasgos finitos)
El niño lee el libro	`Cpl_MS` no tiene regla para un objeto	Objeto directo	Tipo 2 (alcance de la gramática)
El niño y la niña corren	Ninguna regla admite y	Coordinación	Tipo 2 (alcance de la gramática)
El astronauta lee el libro	Palabra ausente del diccionario	Vocabulario	No es de gramática
La mujer vive en la casa de la ciudad	Se acepta con una sola lectura	Ambigüedad	Límite de LL(1)

La discordancia no se detecta como “error de número”, sino como una celda vacía de la tabla. En Los niños corre, AdjMP → eps solo se aplica si el siguiente token está en FOLLOW(AdjMP), y corre (singular) no lo está.

Ambigüedad: Una CFG auxiliar con NP → NP PP y VP → VP PP da exactamente dos árboles para La mujer vive en la casa de la ciudad (la frase se adjunta al nombre o al verbo). El parser LL(1) siempre elige el verbo; spaCy eligió el nombre.

Costo: 24 de las 31 reglas del español son copias por rasgo (una por cada género y número). Añadir persona o tiempo las multiplicaría.

Interpretación: Ningún fallo exige más que una CFG: la concordancia y la subordinación se resolvieron dentro de una. Los demás rechazos se deben al alcance del subconjunto, al diccionario o a LL(1). Afirmar que el español excede toda CFG exigiría reducirlo a un lenguaje

como ww, y este trabajo no lo afirma. Además, spaCy analizó La niño corre sin rechazarla: un parser formal decide pertenencia, un modelo estadístico solo analiza.

Análisis formal de los puntos de falla

Al intentar que la gramática aceptara español descubrimos que no todos los fallos son del mismo tipo. Los separamos en tres, porque cada uno tiene una explicación distinta:

- Límites de las gramáticas libres de contexto (CFG, Tipo 2): cosas que ninguna CFG puede describir, sin importar cuánto se esfuerce quien la escribe.
- Límites del método LL(1): el parser lee de izquierda a derecha, mira una sola palabra por adelantado y debe decidir sin dudar. Hay cosas que una CFG sí puede describir, pero que este método no puede analizar.
- Límites prácticos: se puede escribir, pero hacen falta tantas reglas que deja de ser razonable.

Como veremos, la mayoría de nuestros fallos son de los dos últimos tipos. Solo uno es un límite real de las CFG.

Cuatro resultados formales

Frases con dos significados: el problema de LL(1)

El parser LL(1) usa una tabla que, según la palabra que viene, le dice qué regla aplicar. Cada casilla debe tener una sola respuesta. En La mujer vive en la casa con la hermana, la frase con la hermana puede describir a la casa o al verbo vive. Nuestra gramática acepta la oración, pero con una sola lectura: cuelga ambas frases del verbo. La otra lectura no se puede representar.

Esto no es un descuido nuestro. Es un resultado general: ninguna gramática ambigua es LL(k). Otros parsers, como Earley o CYK, sí pueden guardar las dos lecturas. Por eso el problema es del método LL(1) y no de las CFG.

Nuestro fragmento es más simple de lo que parece

Una forma de medir qué tan potente es una gramática es ver si alguna regla puede producirse a sí misma con texto a ambos lados. Si solo agrega texto de un lado, por ejemplo al final, el lenguaje que genera es regular (Tipo 3), según el teorema de Chomsky (1959).

En nuestra gramática del español ninguna regla cumple esa condición: la subordinada, los adjetivos y los modificadores solo agregan texto por la derecha. Por eso este fragmento, y solo este subconjunto que definimos, es regular (Tipo 3); no se afirma lo mismo del español en general.

Listas que se repiten en el mismo orden: el límite real de las CFG

En Pedro, María y los niños son alto, alta y pequeños respectivamente, el primer adjetivo concuerda con el primer sujeto, el segundo con el segundo, y así. Si anotamos solo el género y el número de cada uno, la secuencia de los sujetos se repite igual en la de los adjetivos. Una CFG funciona como una pila: el último que entra es el primero que sale, lo que sirve para dependencias anidadas. Para copiar una lista hay que sacar los elementos en el mismo orden en que entraron, como en una fila, y una pila no puede hacerlo.

Tres aclaraciones:

- Para una lista de tamaño fijo sí se puede, enumerando las 4^n combinaciones, pero crece muy rápido.
- Las dependencias anidadas sí son libres de contexto. Solo las cruzadas las exceden.
- Que el español tenga dependencias cruzadas es discutido. El caso clásico es el alemán suizo (Shieber, 1985); para el español no hay un caso tan aceptado, y este trabajo no lo afirma.

El costo de repetir reglas

Para que el parser revise la concordancia, cada categoría se repite una vez por cada combinación de rasgos. Con género (2 valores) y número (2 valores) hay 4 versiones de cada regla, y un tercer rasgo con 3 valores las llevaría a 12. Nuestro fragmento tiene 31 no terminales y 57 producciones, y tendría 19 y 33 si el género no se pasara al sintagma verbal.

Agregar persona, tiempo y clases de verbos multiplicaría esas cifras.

Comparación con la salida de NLTK/spaCy sobre las mismas oraciones

Las mismas 13 oraciones (5 simples, 3 intermedias y 5 complejas) pasaron por tres sistemas:

- **Nuestro parser LL(1):** decide si la oración pertenece a la gramática y, si no, dice qué regla o palabra falló.
- **NLTK:** un etiquetador de categorías gramaticales entrenado con 6.030 oraciones. Asigna a cada palabra la etiqueta que más veces vio y, si la palabra es nueva, se guía por su terminación.
- **spaCy:** el modelo “es_core_news_sm 3.8.0”, que entrega etiquetas, dependencias entre palabras y rasgos morfológicos.

Son herramientas de distinto tipo. El parser valida; NLTK y spaCy describen la oración y no emiten un veredicto de "correcta" o "incorrecta". Por eso comparamos en dos tareas: aceptar o rechazar (solo el parser) y etiquetar categorías gramaticales. Repetimos la ejecución en otro equipo con las mismas versiones y los 39 análisis salieron idénticos.

Resultados

Sistema	Etiquetas correctas (de 73)	Errores
spaCy	72 (98,6 %)	**alta** como sustantivo en El niño alta corre
NLTK	70 (95,9 %)	**profesora** como adverbio; **que** como pronombre (2 veces)
Parser propio	69 (94,5 %)	4 palabras que no están en su vocabulario (**y**, **astronauta**, **leyó** y **el punto**)

Sobre la tarea de aceptar o rechazar, el parser acierta las 13 oraciones frente a las etiquetas del subconjunto, pero ese resultado es circular: las etiquetas se definieron a partir de lo que la gramática cubre.

Qué muestran las diferencias

1. El parser detecta errores de concordancia; spaCy y NLTK no. Las cinco oraciones incorrectas (*El niños corre, La niño corre, Los niños corre, El niño alta corre, El profesor sabe que los niños lee el libro*) fueron rechazadas por el parser, con un mensaje que señala el punto exacto, por ejemplo "se esperaba SUST_MS pero llegó 'niños'". spaCy y NLTK las analizaron todas sin objeción. En *Los niños corre*, la propia morfología de spaCy marca el sujeto como plural y el verbo como singular, pero no lo presenta como un error. Es decir, tiene el dato y no lo usa para validar.

2. spaCy y NLTK cubren lo que el parser no. El parser rechazó cinco oraciones correctas: con objeto directo, con coordinación, con una palabra nueva (*astronauta*), en pasado y con punto final. Todas quedan fuera de la gramática o del vocabulario por diseño, y spaCy las procesó sin problema. Ante una palabra nueva, el parser se detiene, mientras que NLTK y spaCy hacen una estimación.

3. Los modelos estadísticos se adaptan al error en lugar de señalarlo. En *El niño alta corre*, spaCy etiquetó *alta* como sustantivo, es decir, "corrigió" la oración para que tuviera sentido. El parser, en cambio, la rechazó porque *alta* no concuerda con *niño*. Los errores de NLTK son de otro tipo: etiqueta cada palabra por separado, sin mirar el contexto. Por eso marcó *profesora* como adverbio y *que* como pronombre, que es su uso más frecuente, aunque en estas oraciones es una conjunción.

4. La ambigüedad se resuelve distinto. En *La mujer vive en la casa de la ciudad*, nuestro parser genera un solo árbol, con las dos frases preposicionales colgando del verbo. Una CFG auxiliar con el ChartParser de NLTK genera dos árboles, uno con *de la ciudad* describiendo a *casa* y otro describiendo a *vive*. spaCy produce uno solo y elige la primera lectura (*ciudad* depende de *casa*).

Los dos enfoques son complementarios. La gramática formal da control y explicación: rechaza con razón lo que no cumple una regla y dice por qué, pero solo conoce lo que se le enseñó. Los modelos estadísticos dan cobertura y robustez: procesan casi cualquier oración, pero no garantizan que sea correcta ni explican sus decisiones.

Referencias

Alfonseca Cubero, E., Alfonseca Moreno, M. y Moriyón, R. (2007). *Teoría de autómatas y lenguajes formales*. McGraw-Hill.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory, 2*(3), 113–124.

Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control, 2*(2), 137–167.

Louden, K. C. (2004). *Construcción de compiladores: principios y práctica*. Thomson.

Shieber, S. M. (1985). Evidence against the context-freeness of natural language. *Linguistics and Philosophy, 8*(3), 333–343.